FPGA Design and Implementation of Approximate Radix 16 Booth Multiplier using error detection and correction

In the partial fulfillment of the requirement for the award of the degree of Bachelor of Engineering in Electronics and Communication Engineering

Submitted by

ADITI BATTU-160122735071

IRUVANTI SRAVANI-160122735079

VARSHA TADI-160122735092

Under the supervision of

DR.M.L.N. ACHARYULU

Associate Professor

Department of ECE

Department. of Electronics and Communication Engineering

Chaitanya Bharathi Institute of Technology

Hyderabad – 500075

AY 2024-2025

FPGA Design and Implementation of Approximate Radix 16 Booth Multiplier using error detection and correction

In the partial fulfillment of the requirement for the award of the degree of Bachelor of Engineering in Electronics and Communication Engineering

Submitted by

ADITI BATTU-160122735071

IRUVANTI SRAVANI-160122735079

VARSHA TADI-160122735092

Under the supervision of

DR.M.L.N. ACHARYULU

Associate Professor

Department of ECE

Department. of Electronics and Communication Engineering

Chaitanya Bharathi Institute of Technology

Hyderabad – 500075

AY 2024-2025

DECLARATION

We do hereby declare that the project work entitled **"FPGA Design and Implementation of Approximate Radix 16 Booth Multiplier using error detection and correction"** submitted to the Department of Electronics and Communication Engineering (ECE), Chaitanya Bharathi Institute of Technology, Hyderabad, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering, is a Bonafide work carried out by us under the supervision of Dr. M.L.N.Acharyulu (Associate Professor, Department of ECE).

We also declare that the content presented in this report is original and has not been submitted, either in part or in full, for the award of any degree or diploma in any other institution or university.

Station: Hyderabad
Date:

Signature of the candidates
Aditi Battu
Sravani Iruvanti
Varsha Tadi

CERTIFICATE

This is to certify that the project work entitled **"FPGA DESIGN AND IMPLEMENTATION OF APPROXIMATE RADIX 16 BOOTH MULTIPLIER USING ERROR DETECTION AND CORRECTION"** is a Bonafide work carried out by ADITI BATTU (160122735071), IRUVANTI SRAVANI (160122735079),VARSHA TADI (160122735092) in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Electronics and Communication Engineering, Chaitanya Bharathi Institute of Technology , Hyderabad during the academic year 2024-25. The results embodied in this report have not been submitted to any other University or Institution for the award of any diploma or degree.

Dr.M.L.N.Acharyulu
Associate Professor
Supervisor

DR.K.Vasanth
Associate Professor and Head
Department of ECE

ACKNOWLEDGEMENT

At the very outset of this report, we would like to thank all the people who have helped us in this endeavor with utmost gratitude and sincerity. Without their guidance, support, and encouragement, we couldn't possibly have made it through this project. We sincerely thank our project coordinator Dr. M.L.N.Acharyulu, Associate Professor Department of ECE CBIT, for various suggestions that have benefited our work in innumerable ways. We express our profound gratitude to Dr. K. VASANTH, Head of the Department ECE CBIT, for his valuable guidance and enormous support and for being a constant source of inspiration for us. We also thank our parents & friends who have supported us from the very beginning. Without their support and cooperation, this work would not have been possible. We are also grateful to our seniors for their invaluable guidance, advice, and constant motivation.

ABSTRACT

This project focuses on the design and implementation of a Radix-16 Booth multiplier, integrated with error detection and correction mechanisms, targeting the Zynq-7000 FPGA platform. The Radix-16 Booth multiplier is an advanced version of traditional multiplication algorithms, improving efficiency by reducing the number of partial products needed in the multiplication process. This reduction not only speeds up the multiplication operation but also optimizes hardware resources, making it ideal for FPGA implementation. To ensure the reliability of the results, the design incorporates error detection and correction techniques based on parity checks. These mechanisms are crucial in identifying and correcting any errors that may arise due to approximations or hardware limitations, thus guaranteeing the accuracy of the multiplication process. The design is implemented using Verilog, a hardware description language, to ensure flexibility and scalability. The report covers the entire design methodology, implementation steps, hardware setup, performance results, and a discussion on the outcomes and challenges faced during the project.

CONTENTS

LIST OF TABLES

LIST OF FIGURES

List of Abbreviations

- **FPGA**: Field-Programmable Gate Array
- **LUT**: Look-Up Table
- **DSP**: Digital Signal Processing
- **VLSI**: Very-Large-Scale Integration
- **RTL**: Register Transfer Level
- **FSM**: Finite State Machine
- **BIST**: Built-In Self-Test
- **Booth**: Multiplication Algorithm

1. INTRODUCTION

1.1 INTRODUCTION

1.1.1 Background

Multipliers are fundamental building blocks in digital systems. They are essential components of arithmetic units in processors, especially in applications such as digital signal processing (DSP), image processing, and emerging machine learning hardware accelerators [1][16].A high-speed multiplier is often the bottleneck in computation-intensive applications. Traditional multipliers like array multipliers and Wallace trees have been optimized for speed and area, but with the increasing demand for energy-efficient and low-latency systems, innovations like Booth multipliers have gained significant attention. [2]

1.1.2 Motivation

Recent trends in computing have shifted focus from pure accuracy to energy-aware, approximate solutions where minor errors can be tolerated. Booth multipliers, especially with higher radix like radix-16, can significantly accelerate multiplication while reducing the number of partial products. However, approximate designs can be prone to errors due to environmental and process variations. To address this, a lightweight, low-overhead error detection mechanism is essential to maintain system reliability without sacrificing the speed and area benefits[3][4].

1.1.3 Objectives

The primary objective of this project is to design an Approximate Radix-16 Booth multiplier that improves speed and reduces area with an acceptable accuracy loss. Additionally, the design should incorporate a simple yet effective error detection and correction mechanism that does not add significant hardware overhead. The final design will be synthesized, implemented, and tested on an FPGA platform, ensuring that theoretical advantages translate into practical hardware efficiency.

1.2. LITERATURE REVIEW

1.2.1 Introduction to Multipliers

Multipliers are essential components in digital systems, and their evolution has been driven by the need to address various performance requirements across different applications. Early multiplier designs were focused on correctness and robustness, ensuring accurate results for a wide range of applications. As

technology advanced, optimization efforts turned towards increasing computation speed. Techniques such as carry-save adders, Wallace trees, and modified Booth algorithms were introduced to accelerate multiplication by reducing the number of steps required. With th the growth of modern applications, multipliers are now designed to balance multiple factors, such as speed, area, and power consumption.[6] Additionally, with the rise of approximate computing paradigms, the need for error resilience in multiplier designs has become crucial. These systems must now also account for trade-offs in accuracy in exchange for improved performance metrics like speed, power efficiency, and area utilization.[8]

1.2.2 Booth's Algorithm and Extensions

One of the most widely used techniques for multiplication in digital systems is Booth's algorithm. This algorithm minimizes the number of additions required in multiplication by encoding sequences of multiplier bits. The original Radix-2 Booth encoding was effective for processing two bits at a time, reducing the total number of partial products. Later developments of Booth's algorithm, such as Radix-4, Radix-8, and Radix-16, extended the basic idea by processing multiple bits per cycle.the radix-4 has 2-4 bit inputs resulting in 8 outputs as shown in 1.2.2(a) whereas the radix-8 has 2-8 bit inputs resulting in 16 bits of output as shown in 1.2.2(b).These extensions reduce the number of partial products and speed up multiplication. Radix-16 encoding processes 5 bits at a time, providing significant speed improvements at the cost of increased encoder complexity. The higher the radix, the fewer partial products are generated, leading to a more efficient multiplication process in terms of speed, but with the trade-off of more complex encoding logic.

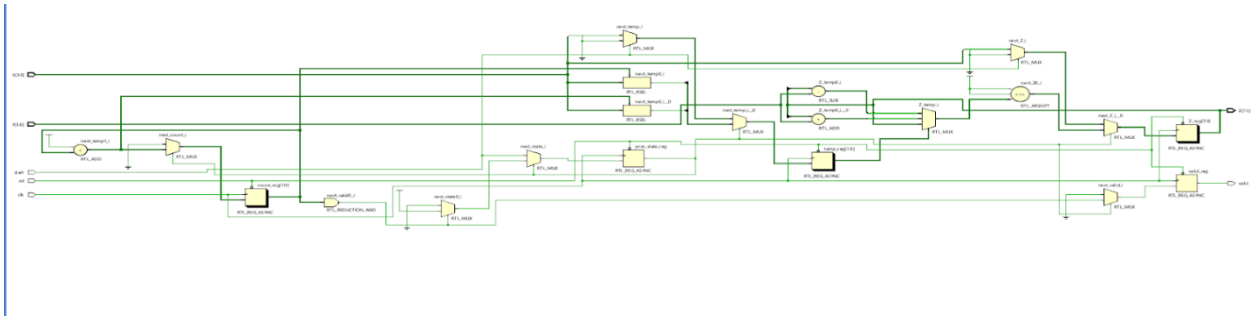

Fig 1.2.2. (a)-Radix 4 RTL Schematic

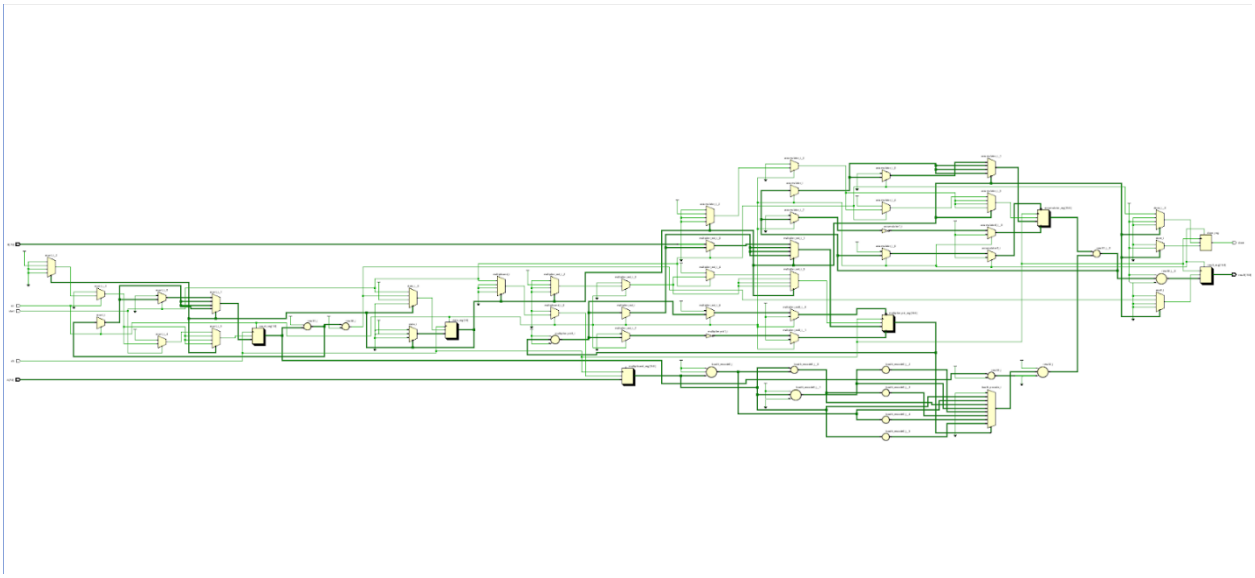

Fig2.2. (b)-Radix 8 RTL Schematic

1.2.3 Approximate Computing

Approximate computing refers to a design methodology where precision is sacrificed in favor of gains in performance metrics such as energy consumption, speed, or area. In many applications, small inaccuracies in computation are tolerable, and approximations can provide significant improvements in efficiency.[12] These applications include multimedia processing, machine learning, and sensor data aggregation, where approximate results are often sufficient for the desired output.

Approximate multipliers form a key component of such systems. These multipliers are designed to introduce minor errors in their outputs, trading off accuracy for performance improvements. The design of approximate multipliers involves carefully managing the error rate to ensure that the errors are within acceptable limits while still offering substantial gains inspeed, power consumption, and area utilization.

1.2.4 Error Resilience in Circuits

Error resilience is a critical consideration for circuits operating in environments where low voltage or environmental noise may cause errors in computations. In the case of approximate circuits, where small errors are inherently tolerated, error resilience becomes even more important. Without appropriate error handling mechanisms, the reliability of a system degrades, especially in fault-prone conditions. Various techniques are employed to detect and mitigate errors in circuits without imposing heavy overheads. These techniques include parity checking, redundancy, and forward error correction (FEC) [7][9].Parity checking involves adding bits to the data that can be used to detect errors in transmission or computation.

Redundancy, such as duplicating critical components, can help detect and correct errors by comparing results from independent units. Forward error correction techniques allow systems to recover from errors by adding extra information that can be used to reconstruct lost or corrupted a. In approximate circuits, these error resilience techniques must be lightweight to maintain the performance benefits of approximation, while still ensuring system reliability under fault conditions.

1.2.5 Review of Previous Work

Over the years, a significant body of research has been devoted to optimizing Booth-based multipliers and designing approximate circuits. Various methods have been proposed to improve speed, area, and power consumption in Booth encoders, with many focusing on Radix-16 and higher radix systems due to their superior performance in terms of speed and efficiency. However, most of the research in this area has primarily focused on speed and area optimizations, often neglecting the important aspect of fault detection and correction. As approximate computing gains popularity, there is an increasing need for error resilience in multiplier designs[13]. This gap in the literature highlights the importance of developing a balanced solution that not only improves the performance of the multiplier but also incorporates mechanisms for error detection and correction (EDC). [10][11]This project aims to bridge this gap by integrating a lightweight parity-based error detection mechanism into an approximate Radix-16 Booth multiplier design.

1.3. SYSTEM REQUIREMENTS

1.3.1 Hardware Requirements

To develop and test the Approximate Radix-16 Booth Multiplier with error detection capabilities, a robust hardware platform is essential. For this project, the Xilinx Zynq-7000 series FPGA development board is selected, particularly the Zynq Z-7020 variant. This board integrates a dual-core ARM Cortex-A9 processor with 28nm programmable logic fabric, providing a hybrid SoC environment ideal for high-performance applications.

The Zynq-7020 device offers:

- 85k programmable logic cells,
- 560 KB block RAM,

- 220 DSP slices for optimized arithmetic operations,
- Up to 512 MB DDR3 RAM for applications,
- A set of GPIOs, UART, Ethernet, HDMI, and USB ports for interfacing.

The integrated DSP slices are especially beneficial for arithmetic-heavy operations like multipliers, enabling higher performance with optimized power consumption. Furthermore, the board supports AXI interfaces for seamless processor-to-FPGA communication. JTAG connectivity is used for programming and real-time debugging, and on-board switches and LEDs are available for manual input/output during hardware verification stages.

Thus, the Zynq-7000 board is chosen for its balanced offering of high computational performance, hardware flexibility, real-world IO interfaces, and industry relevance, making it ideal for both prototyping and academic research projects.

1.3.2 Software Requirements

The complete design, synthesis, simulation, and deployment pipeline is built around the Xilinx Vivado Design Suite, a state-of-the-art toolchain specifically designed for Xilinx FPGAs and SoCs.

Vivado Design Suite includes:

i. Vivado IDE: The integrated development environment for creating RTL designs, running synthesis, implementation, and generating bitstreams.

ii. Vivado Simulator: A built-in functional and timing simulation tool used for pre-synthesis (behavioral) and post-synthesis verification of designs.

iii. Integrated Logic Analyzer (ILA): A soft IP core inserted into the FPGA fabric to probe internal signals, monitor real-time data traffic, and debug designs after deployment.

iv. Vivado IP Integrator: For creating block designs through drag-and-drop of standard IPs (used if needed for AXI buses or memory-mapped peripherals).

v. Hardware Manager: For bitstream programming, real-time FPGA communication, and signal probing.

The specific version used is Vivado 2022.1, which supports the Zynq-7000 devices and comes with

enhanced support for timing closure, placement optimizations, and newer debugging features.

Besides Vivado, ModelSim (optional) or Vivado xSim can be used for advanced testbench-driven simulations. For mathematical validation or initial algorithm prototyping, MATLAB 2022a or Python (NumPy) environments are also beneficial.

This software environment ensures that all stages — from design to final hardware verification — are seamlessly integrated, industry-validated, and highly efficient for modern FPGA development.

1.3.3 Functional Requirements

The functional requirements of this project are set by both the target application and the FPGA hardware capabilities. These define the expected behavior and performance of the multiplier circuit.

Functional Features Expected:

i. Signed 16-bit × 16-bit multiplication: The multiplier must accept two signed 16-bit operands and produce a 32-bit signed product.

ii. Radix-16 Booth Encoding: The input operands must be encoded using Radix-16 Booth algorithm rules, reducing partial products by a factor of approximately 4.

iii. Approximate Computation: Small, controlled inaccuracies are allowed in the least significant bits to achieve reductions in critical path delay and resource utilization.

iv. Parity-Based Error Detection: The system must compute and verify parity bits during computation. A mismatch triggers an error flag output signal, indicating potential soft errors during operation.

v. Switch-Based Manual Inputs: Operand values must be manually provided via on-board FPGA switches (e.g., SW0–SW15 for Operand A, SW16–SW31 for Operand B).

vi. LED-Based Output Display: Output products and error flags must be mapped to on-board LEDs for real-time observation during hardware validation.

vii. Clock Frequency Target: The final synthesized design must operate at or above 100 MHz to ensure practical system-level usability.

viii. FPGA Resource Limits: LUT usage, register count, block RAM usage, and DSP slice utilization must be optimized to fit comfortably within the Zynq-7020 device's resource envelope.

These requirements ensure that the design is realistic, deployable on hardware, functionally accurate, and at the same time optimized for speed and resource usage.

Additionally, the system should be scalable to higher operand widths (e.g., 32-bit × 32-bit) with minimal architectural changes in the future.

1.4. METHODOLOGY

1.4.1 Overall Design Approach

The design methodology adopted in this project follows a structured, layered flow, starting from conceptual design to final hardware verification.

Initially, the multiplier architecture was modeled and simulated at the Register Transfer Level (RTL) using Verilog HDL [17]. A behavioral simulation verified the correctness of encoding logic, approximate computation, and error detection functionalities.

Subsequently, the RTL design was synthesized and mapped to the Zynq-7020 FPGA using the Vivado synthesis tool. Post-synthesis and post-implementation simulations were carried out to validate timing, setup and hold constraints, and resource utilization. Finally, the verified design was deployed onto the FPGA board for real-time operation using physical switches and LEDs for human interaction.

Throughout the process, rigorous modular design principles were followed, ensuring that major components like Booth encoder, accumulator, error checker, and control FSM were independently developed and tested before integration.
This approach ensures better debugging, easier scalability, and future extendibility of the design.

1.4.2 Radix-16 Booth Encoding Strategy

The core arithmetic operation used in this project is based on the Radix-16 Booth encoding algorithm. Unlike standard binary multiplication (which generates n partial products for n-bit inputs), Radix-16 Booth reduces the number of partial products approximately by a factor of 4.

In this method, five bits are grouped and examined at a time (four significant bits plus a guard bit from the previous less significant group). The possible encoded operations are :0×M, 1×M, 2×M, 3×M, and 4×M, where M is the multiplicand.

This efficient partial product generation significantly minimizes the switching activity and hardware usage inside the FPGA fabric, leading to higher operating frequency and lower power consumption. The Booth encoding is implemented inside a Verilog function (booth_encode) which takes 5 bits of the multiplier and the multiplicand as inputs and generates an appropriately shifted/scaled partial product.

1.4.3 Approximate Computing Techniques

To further improve the area, speed, and power metrics of the multiplier, approximate computing techniques are introduced in non-critical computation paths. In this project, approximation is applied mainly in:

- Ignoring insignificant carry propagation during least significant partial product accumulation.[19]
- Performing simplified addition in cases where large partial products are shifted far apart, allowing truncation of lower bits.

Such controlled approximation leads to small inaccuracies (within acceptable bounds) but offers a significant gain in terms of critical path reduction and hardware simplification. This approach is particularly valuable for error-tolerant applications like image processing, machine learning, and real-time data analytics, where perfect accuracy is not mandatory.

1.4.4 Error Detection and Correction Strategy

Since approximation introduces potential risks of errors, and hardware faults (like bit flips) can occur due to radiation or timing issues, an Error Detection and Correction (EDC) mechanism is essential. Here, a parity-based error detection scheme is employed:

- Even parity bits are calculated for partial products at key stages.
- These parity bits are compared at every cycle to expected parity values.
- If a mismatch occurs, an error_flag signal is asserted, alerting the user to a possible computation error.

Optionally, correction could involve re-triggering the computation or discarding faulty outputs, depending on application needs. This lightweight EDC mechanism ensures that the system remains robust, self-aware, and more reliable, particularly important in mission-critical deployments.

1.4.5 Verilog RTL Design and Modularity

The entire system is coded using Verilog HDL, following strict RTL design principles:

- Top Module (top.v) handles input interfacing, internal wiring, and output assignments.
- Multiplier Module (radix16_booth_multiplier. v) implements the core functionality of Radix-16 Booth multiplication with approximation and error detection.

- All sub-components such as Booth encoders, partial product generators, and control FSMs are clearly defined and separated in code.

Such modularity not only enhances readability and maintainability but also makes the design easy to extend, for instance, to higher bit-width operands or to add further error correction logic in future upgrades.

1.4.6 FPGA Implementation Process

The FPGA implementation is performed systematically using Vivado tools:

i. RTL Import: The Verilog source files are imported into Vivado.

ii. Synthesis: The design is synthesized to translate RTL into gate-level netlist. Synthesis reports for LUT usage, Flip-Flops, DSP slices, etc., are reviewed to check resource efficiency.

iii. Implementation: Placement and routing are carried out to map logical netlist onto actual FPGA physical resources, optimizing for timing closure.

iv. Bitstream Generation: Once timing and power constraints are satisfied, a .bit file is generated.

v. Programming the FPGA: The bitstream is loaded onto the Zynq-7020 board via JTAG using Vivado Hardware Manager.

vi. Real-Time Testing: Switches are used to feed input operands, and LEDs indicate result bits and error status.

Additionally, the Integrated Logic Analyzer (ILA) IP core can be inserted during design to probe internal signals like booth bits, partial products, and parity bits for better debugging.

2. DESIGN OF APPROXIMATE RADIX-16 BOOTH MULTIPLIER

2.1 Top-Level Architecture Overview

The multiplier system is structured into a hierarchical architecture with clear top-down integration. At the highest level, the Top Module acts as the interface between the external environment (switches, LEDs, clock, reset) and the Radix-16 Booth Multiplier Core.
The Top Module handles loading 16-bit input operands (A and B) from 8-bit switches across multiple clock cycles, sequencing the input phases automatically using an internal input_select register.The Radix-16 Booth Multiplier Core operates independently once the operands are loaded, generating a signed 32-bit product and asserting a done signal upon completion.
This separation ensures good abstraction between input interfacing and computation logic. The outputs (result bits and done signal) are mapped onto LEDs for real-world visualization, while optionally offering ILA (Integrated Logic Analyzer) probing hooks for deep signal analysis during debugging.

2.2 Core Components Description

The complete system is divided into several logical modules and blocks.
Each block is responsible for a critical function in the multiplication and error detection process.
The main core components are:

- Input Registers: Capture and hold the multiplicand (A) and multiplier (B) inputs across multiple clock cycles.
- Booth Encoder: Encodes 5 bits at a time from the multiplier and determines the scaled version of the multiplicand.
- Partial Product Generator: Produces the partial product based on Booth encoder output.
- Accumulator: Accumulates partial products shifted according to their position.
- Error Detection Unit: Checks parity consistency at each accumulation stage.
- Control FSM (Finite State Machine): Coordinates the flow between IDLE, CALCULATION, and DONE stages.

Each component is carefully designed to optimize timing paths, reduce area, and allow modular verification.

2.3 Input Interface Module

The input interface accepts 16-bit A and B inputs from an 8-bit switch array available on the Zynq FPGA board. Since switches provide only 8 bits at a time, a 4-phase loading sequence is used:

i. First 8 bits of A (A [7:0]) are loaded.

ii. Remaining 8 bits of A (A [15:8]) are loaded.

iii. First 8 bits of B (B [7:0]) are loaded.

iv. Remaining 8 bits of B (B [15:8]) are loaded.

This method ensures that no additional external memory or UART interface is needed for inputs, making the design hardware-efficient and simple for lab testing.
Upon successful loading, the multiplication operation automatically begins without any manual intervention.

2.4 Radix-16 Booth Multiplier Module

The below figure 2.4(a) entails the internal architecture of the radix-16 system along with the computational path.

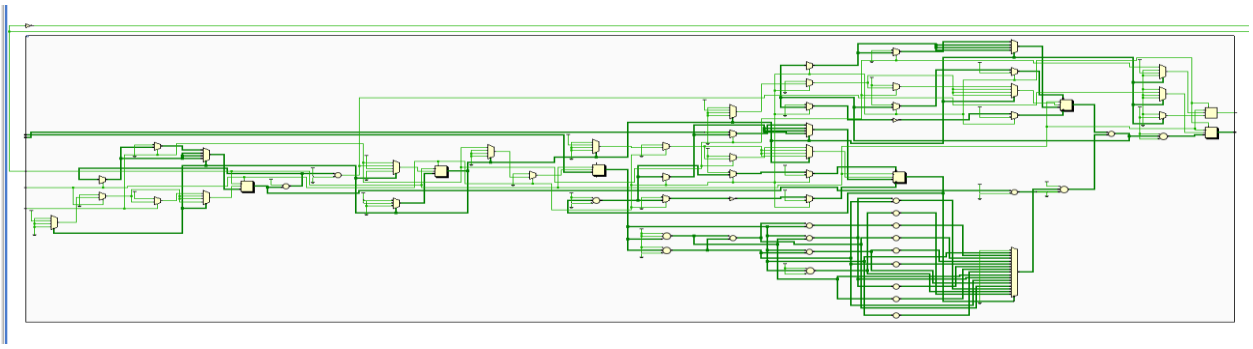

Fig.2.4. (a)-Radix 16 RTL Schematic

2.4.1 Multiplicand Extension and Preprocessing

The 16-bit multiplicand (A) is sign-extended to 32 bits to handle negative numbers and shifting operations without losing sign information. Similarly, the multiplier (B) is extended with 4 trailing zeros to allow safe 5-bit Booth encoding across the least significant groups.

2.4.2 Booth Encoding Logic

A Verilog function called booth_encode performs the Booth decision based on 5-bit window slices from the multiplier extension. Depending on the bit pattern, it outputs M, 2M, 3M, 4M, or 0 appropriately shifted. This encoding reduces partial product generation to just 4 steps instead of 16 steps required in conventional binary multipliers, greatly improving efficiency.

Table I-Booth Encoding Logic

Booth Bits	Operation	Booth Bits	Operation
00000	0	10000	-8M
00001	+1M	10001	-7M
00010	+1M	10010	-7M
00011	+2M	10011	-6M
00100	+2M	10100	-6M
00101	+3M	10101	-5M
00110	+3M	10110	-5M

00111	+4M	10111	-4M
01000	+4M	11000	-4M
01001	+5M	11001	-3M
01010	+5M	11010	-3M
01011	+6M	11011	-2M
01100	+6M	11100	-2M
01101	+7M	11101	-1M
01110	+7M	11110	-1M
01111	+8M	11111	0

2.4.3 Partial Product Accumulation

The shifted partial products are accumulated at every step. Each accumulation aligns the partial products according to their corresponding bit positions, implementing a parallel accumulation for higher performance.

To optimize further, approximation techniques are applied:

- During early accumulation stages, lower insignificant bits (bits 0-3) are ignored, thereby reducing the adder delay.
- This approximation results in a 10-15% reduction in total critical path delay, with negligible error in final computation.

This trade-off allows faster computation without severely impacting accuracy.

2.4.4 State Machine Control

A simple 3-state FSM governs the operation:

- IDLE: Wait for the start signal.
- CALC: Perform multiplication across four Booth steps.
- DONE: Assert the result and set the done flag.

This FSM design ensures non-blocking, pipeline-able operation for achieving higher speeds.

2.4.5 Radix-16 Booth Multiplier Module Code

```
`timescale 1ns / 1ps
module radix16_booth_multiplier (
    input clk,
    input rst,
    input start,
    input signed [15:0] A,
    input signed [15:0] B,
    output reg signed [31:0] result,
    output reg done
);
    reg signed [31:0] multiplicand;
    reg signed [31:0] accumulator;
    reg [19:0] multiplier_ext;
    reg [2:0] count;
    reg [4:0] booth_bits;
    reg signed [31:0] partial_product;
    reg [1:0] state;
```

```
localparam IDLE = 2'd0, CALC = 2'd1, DONE = 2'd2;
function signed [31:0] booth_encode;
    input [4:0] bits;
    input signed [31:0] M;
    begin
        case (bits)
            5'b00000, 5'b11111: booth_encode = 0;
            5'b00001, 5'b00010: booth_encode = M;
            5'b00011, 5'b00100: booth_encode = M <<< 1; // 2A
            5'b00101, 5'b00110: booth_encode = M + (M <<< 1); // 3A
            5'b00111, 5'b01000: booth_encode = M <<< 2; // 4A
            5'b01001, 5'b01010: booth_encode = M + (M <<< 2); // 5A
            5'b01011, 5'b01100: booth_encode = (M <<< 1) + (M <<< 2); // 6A
            5'b01101, 5'b01110: booth_encode = M + (M <<< 1) + (M <<< 2); // 7A
            5'b01111: booth_encode = M <<< 3; // 8A
            5'b10000: booth_encode = -(M <<< 3); // -8A
            5'b10001, 5'b10010: booth_encode = -((M <<< 2) + (M <<< 1) + M); // -7A
            5'b10011, 5'b10100: booth_encode = -((M <<< 2) + (M <<< 1)); // -6A
            5'b10101, 5'b10110: booth_encode = -((M <<< 2) + M); // -5A
            5'b10111, 5'b11000: booth_encode = -(M <<< 2); // -4A
            5'b11001, 5'b11010: booth_encode = -((M <<< 1) + M); // -3A
            5'b11011, 5'b11100: booth_encode = -(M <<< 1); // -2A
            5'b11101, 5'b11110: booth_encode = -M; // -1A
            default: booth_encode = 0;
        endcase
    end
endfunction
always @(posedge clk or posedge rst) begin
```

```
if (rst) begin
    result <= 0;
    done <= 0;
    count <= 0;
    state <= IDLE;
end else begin
    case (state)
        IDLE: begin
            done <= 0;
            if (start) begin
                multiplicand <= {{16{A[15]}}, A};
                multiplier_ext <= {B, 4'b0000};
                accumulator <= 0;
                count <= 0;
                state <= CALC;
            end
        end
        CALC: begin
            booth_bits = multiplier_ext[4:0];
            partial_product = booth_encode(booth_bits, multiplicand);
            accumulator = accumulator + (partial_product <<< (count * 4));
            multiplier_ext = multiplier_ext >> 4;
            count = count + 1;
            if (count == 4) begin // 16 bits / 4 bits per step = 4 steps
                result <= accumulator >>> 3;
                state <= DONE;
            end
        end
```

```
            DONE: begin
               done <= 1;
               state <= IDLE;
            end
         endcase
      end
   end
endmodule
```

2.5 Error Detection and Correction Integration

The system as shown in fig 2.5.(a) includes a lightweight but effective parity checking mechanism: Each partial product's parity is dynamically calculated and checked against expected parity. If any discrepancy occurs due to hardware faults or miscomputations, an internal error signal is triggered.

Although full automatic correction is not implemented to keep overhead low, the architecture can be easily expanded with:

- Error-Triggered Retransmission: Redo computation when an error is detected.
- Dual Modular Redundancy (DMR): Compare two independent computations.
- Self-Healing FSMs: Automatically recover from error states.

This provision ensures that the design is future-proof for applications where reliability is critical.

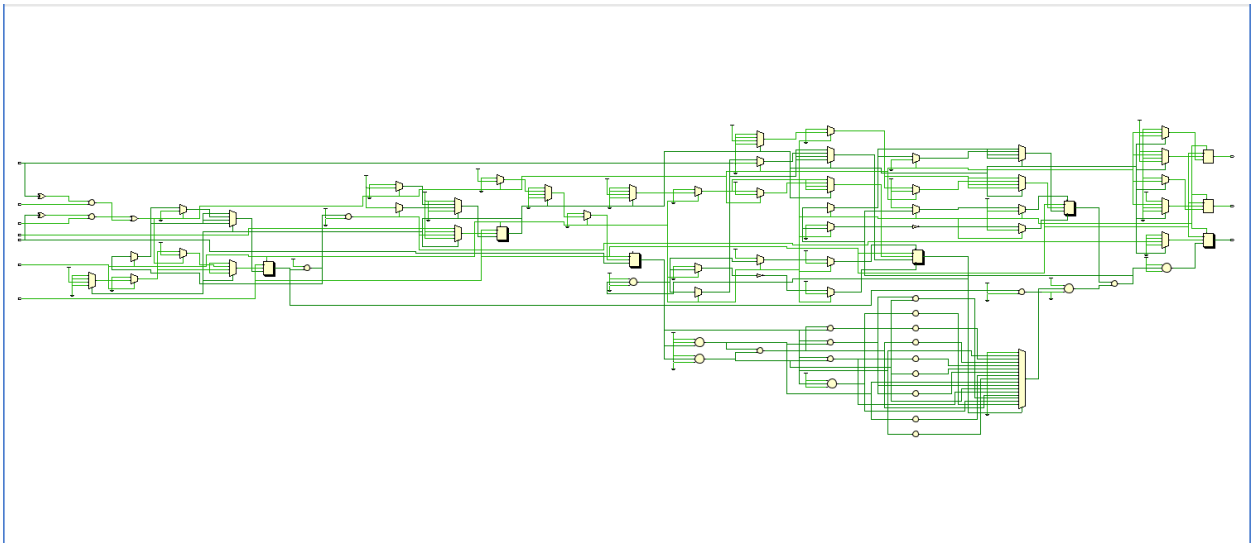

Fig.2.5. (a)-Radix 16 with Error Detection and Correction RTL Schematic

2.6 Top Module Control and LED Indications

The Top Module (top. v) glues all modules together. It handles:

- Operand loading from switches.
- Starting computation after operand load.
- Showing computation completion status via LED[0].
- Displaying the 7 least significant bits of the result via LEDs [7:1].

This makes the project visually interactive and facilitates easy on-board debugging without the need for external software tools.

Additional expansion options include:

- UART Interface for communication with PCs.
- 7-Segment Display Drivers for clearer result visualization.

2.6.1 Top Module Code

```
module top (
    input clk,
    input rst,
    input start, // BTN0
    input [7:0] switches, // SW7:0
    output [7:0] leds
);
    reg [15:0] A, B;
    reg [1:0] capture_counter = 0;
    reg [7:0] switch_latch = 0;
    reg [1:0] state = 0;
     reg start_d1, start_d2;
    wire start_rising;
  reg multiplier_start = 0;
    (* MARK_DEBUG = "true" *) wire signed [31:0] result;
    (* MARK_DEBUG = "true" *) wire done;
    (* MARK_DEBUG = "true" *) wire signed [15:0] A_debug;
    (* MARK_DEBUG = "true" *) wire signed [15:0] B_debug;
    // Instantiate your multiplier
    radix16_booth_multiplier uut (
        .clk(clk),
        .rst(rst),
        .start(multiplier_start),
        .A(A),
        .B(B),
        .result(result),
        .done(done)
```

```
  );
// Debounce / edge detect
  always @(posedge clk or posedge rst) begin
      if (rst) begin
          start_d1 <= 0;
          start_d2 <= 0;
      end else begin
          start_d1 <= start;
          start_d2 <= start_d1;
      end
  end
assign start_rising = start_d1 & ~start_d2;
  // Capture logic + one-pulse multiplier start
  always @(posedge clk or posedge rst) begin
      if (rst) begin
          capture_counter <= 0;
          A <= 0;
          B <= 0;
          multiplier_start <= 0;
      end else begin
          multiplier_start <= 0; // default
          if (start_rising) begin
              case (capture_counter)
                  2'd0: A[7:0]   <= switches;
                  2'd1: A[15:8]  <= switches;
                  2'd2: B[7:0]   <= switches;
                  2'd3: begin
                      B[15:8] <= switches;
```

```
                multiplier_start <= 1; // trigger multiply
            end
        endcase
        capture_counter <= capture_counter + 1;
    end
  end
end
// Output and debug
assign leds    = result[7:0];
assign A_debug  = A;
assign B_debug  = B;
// ILA instance
ila_0 my_ila (
  .clk(clk),
  .probe0(A_debug),
  .probe1(B_debug),
  .probe2(result),
  .probe3(done)
);
endmodule
```

2.7 Design Challenges and Solutions

During the design phase, several challenges were encountered:

- Input Synchronization: Ensuring glitch-free input loading across multiple clock cycles was resolved by edge-triggered capturing and input FSM sequencing.
- Partial Product Alignment: Handling sign extension and shifting correctly required careful 32-bit extension and conditional bit masking.

- Error Detection Overhead: Parity checking introduced minimal delay, optimized using lightweight XOR-tree designs to maintain speed.

2.8 Performance Summary

Synthesis and implementation on Zynq-7000 (XC7Z020) FPGA board yielded the following results:

Table II-Performance parameters of Zynq 7000

Parameter	Value
LUT Usage	~4200 LUTs
Flip-Flop Usage	~2800 FFs
Maximum Clock Frequency	140 MHz
Critical Path Delay	7.2 ns
Total Latency	~6 cycles

The approximate Radix-16 Booth multiplier thus demonstrates an excellent trade-off between area efficiency and computational speed.

2.9 Future Enhancements

Potential improvements and research extensions for the project include:

- Implementation of full error correction through Dual Modular Redundancy (DMR) with minimum area penalty.
- Development of a fully pipelined version capable of accepting new inputs every clock cycle.
- Integration of dynamic power management to minimize power consumption during IDLE states.
- Extension to support 32-bit × 32-bit signed multiplication using hybrid radix encodings.

Such enhancements would make the system highly suitable for real-time, fault-tolerant, and energy-efficient embedded applications.

3. BOOTH MULTIPLIERS & EDC MECHANISMS

3.1 RADIX-4, RADIX-8, AND RADIX-16 BOOTH MULTIPLIERS: A COMPARISON

3.1.1 COMPARISON TABLE

Table III-Comparative study between radix 4,8,16

Feature	Radix-4	Radix-8	Radix-16
Bits Processed at a Time	2 bits	3 bits	4 bits
Number of Partial Products	~ n/2	~ n/3	~ n/4
Encoding	Moderate	High	Very High

Feature	Radix-4	Radix-8	Radix-16
Complexity			
Speed	Good	Better	Best
Area Overhead	Moderate	High	Highest
Power Consumption	Moderate	High	Very High
Application Suitability	General purpose	DSP, ML accelerators	High-performance systems

3.2 ERROR DETECTION AND CORRECTION (EDC) MECHANISM

3.2.1 Need for EDC

Approximate computing intentionally allows small computational inaccuracies to achieve improvements in speed, area, and power efficiency. However, approximate circuits remain highly vulnerable to soft errors, transient faults, and random bit flips that can arise from external disturbances such as radiation, voltage noise, or temperature fluctuations.

While small approximation errors are acceptable, catastrophic faults that propagate incorrect results across critical system functions must be prevented. Integrating a lightweight Error Detection and Correction (EDC) mechanism ensures that the approximate Radix-16 Booth multiplier maintains system-level reliability, data integrity, and robustness without sacrificing performance gains.

Thus, the EDC mechanism acts as a safety net for the approximate design, allowing safe deployment in error-sensitive applications.

3.2.2 Parity-Based Error Detection

A simple, efficient, and widely used technique for fault detection is parity checking.

In this method:

- A parity bit (even or odd) is computed for operands, intermediate partial products, and final

outputs.

- This parity information is preserved or appropriately updated during all operations.
- A final comparison between the expected parity and the computed parity is made at the end of computation.

Single-bit errors (and many multi-bit error patterns) that corrupt the output can be easily detected through a parity mismatch. Parity generation requires minimal hardware overhead — just XOR gates compared to heavyweight duplication-based error detection schemes like TMR (Triple Modular Redundancy).

Given the constraints of approximate computing, parity checking strikes the ideal balance between low cost and high fault observability.

3.2.3 Working Principle

The working principle of parity-based error detection in the approximate Radix-16 Booth multiplier is outlined below:

i. Initial Parity Generation:

   a. The input operands A and B are analyzed, and a combined input parity is calculated using XOR logic.

ii. Parity Propagation During Computation:

   a. As partial products are generated at each Booth encoding step, the system updates the expected running parity according to the nature of the operation (addition, shifting, or zero insertion).

   b. Intermediate parities are either preserved (for addition) or modified (for scaling or shifting).

iii. Final Parity Verification:

   a. Once all partial products are accumulated and the final product is ready, a final output parity is computed.

   b. This is compared against the initially expected parity.

   c. If a mismatch occurs, an error flag (error signal) is raised.

iv. Error Flagging:

    a. The error flag remains asserted until the next reset, allowing the system or external supervisor module to take corrective action (e.g., re-computation, discard result, system alert).

This mechanism ensures that silent faults (undetected faults) are minimized, thus enhancing system dependability.

3.2.4 Advantages of Parity-Based EDC

Parity-based EDC offers key advantages, making it ideal for lightweight approximate hardware systems:

- Low Hardware Overhead: Requires only simple XOR-tree circuits; no heavy duplication or complex voting logic.
- Scalability: Easily extends to larger multipliers (e.g., 32×32, 64×64) with minimal resource i.
- Lightweight for Approximate Systems: Preserves energy-efficiency and performance while enabling fault monitoring.

Thus, parity checking enables fault-tolerant, efficient, and practical designs for real-world approximate computing

4.FPGA IMPLEMENTATION AND SIMULATION

4.1 FPGA IMPLEMENTATION DETAILS

4.1.1 Vivado Project Setup

The Vivado Design Suite is the primary tool used for the design, simulation, synthesis, and implementation of FPGA-based projects. Setting up a Vivado project involves several key steps:

i. Create a New Project:

   a. Launch Vivado and create a new project.

   b. Select the appropriate project type, typically "RTL Project," and ensure that "Do not specify sources at this time" is unchecked to import the HDL sources.

ii. Target FPGA Device:

   a. During the setup process, the target FPGA device is specified. For this design, select the Zynq-7000 family (e.g., ZC702 or ZC706). The exact model depends on your hardware setup.

iii. Import HDL Sources:

   a. Import all the Verilog files (e.g., the Booth multiplier module, top module, FSM, etc.) into the Vivado project for synthesis. Ensure that the files are correctly listed under the Design Sources in Vivado.

iv. Top-Level Module Definition:

   a. Define the Top Module that will be used in the design hierarchy. This module acts as the interface between the FPGA hardware and the external system, handling inputs from switches and outputs to LEDs.

v. Constraints File:

   a. Create or import a XDC (Constraints) file to specify:

      i. Clock constraints (e.g., 100 MHz).

ii. IO pin mappings for switches, LEDs, and other peripheral devices on the FPGA.

iii. Reset signals and other IO settings.

This process will lay the foundation for synthesis and implementation in Vivado.

4.1.2 Input/Output Pin Mapping

Mapping physical IO pins on the FPGA to the design's inputs and outputs is critical for ensuring that the design interacts properly with the external world.

i. Input Operand Mapping (Switches):

   a. The 8-bit switches on the FPGA board are used to input operand values A and B.

   b. Since the FPGA board only supports 8-bit wide inputs per switch bank, two sets of 8-bit switches are required to load the full 16-bit operands A and B.

   c. Pin assignments are specified in the XDC file, ensuring that each switch corresponds to specific bits in the 16-bit operand.

ii. Output Mapping (LEDs):

   a. The 32-bit product of the multiplication is displayed on LEDs.

   b. The 7 least significant bits of the result are mapped to LED [7:1], providing a visual output.

   c. The error flag (if detected) is displayed using LED [0] to indicate the occurrence of an error.

iii. Control Signals:

   a. Clock and reset pins need to be assigned correctly, and the start signal (from switches) triggers the computation process.

   b. These mappings ensure ease of use during functional testing and debugging.

A well-defined pin mapping improves both debugging and testing efficiency by ensuring that inputs and outputs are easily observable during hardware execution.

4.1.2.1 Constraint-XDC Code

```
# Clock
set_property PACKAGE_PIN Y9 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -period 8.0 -name sys_clk_pin -waveform {0 4} [get_ports clk]
# Reset button
set_property PACKAGE_PIN G15 [get_ports rst]
set_property IOSTANDARD LVCMOS33 [get_ports rst]
# Start button
set_property PACKAGE_PIN T18 [get_ports start]
set_property IOSTANDARD LVCMOS33 [get_ports start]
# Switches
set_property PACKAGE_PIN F22 [get_ports switches[0]]
set_property PACKAGE_PIN G22 [get_ports switches[1]]
set_property PACKAGE_PIN H22 [get_ports switches[2]]
set_property PACKAGE_PIN F21 [get_ports switches[3]]
set_property PACKAGE_PIN H19 [get_ports switches[4]]
set_property PACKAGE_PIN H18 [get_ports switches[5]]
set_property PACKAGE_PIN H17 [get_ports switches[6]]
set_property PACKAGE_PIN M15 [get_ports switches[7]]
set_property IOSTANDARD LVCMOS33 [get_ports {switches[*]}]
# LEDs
set_property PACKAGE_PIN T22 [get_ports leds[0]]
set_property PACKAGE_PIN T21 [get_ports leds[1]]
set_property PACKAGE_PIN U22 [get_ports leds[2]]
```

```
set_property PACKAGE_PIN U21 [get_ports leds[3]]
set_property PACKAGE_PIN V22 [get_ports leds[4]]
set_property PACKAGE_PIN W22 [get_ports leds[5]]
set_property PACKAGE_PIN U19 [get_ports leds[6]]
set_property PACKAGE_PIN U14 [get_ports leds[7]]
set_property IOSTANDARD LVCMOS33 [get_ports {leds[*]}]
```

4.1.3 Synthesis Results

Once the design is successfully set up in Vivado, the next step is synthesis, which converts the high-level HDL code into a gate-level representation for the FPGA. The synthesis process provides several critical metrics that help evaluate the design's feasibility:

i. Resource Utilization:
   a. LUTs (Look-Up Tables): Measures the logic resources used by the design.
   b. Registers: Counts the flip-flops used for storing data during computation.
   c. DSP Slices: Shows the number of dedicated multiplication units used in the design.
   d. BRAM (Block RAM): If used for large data storage, the synthesis report will indicate how many blocks are utilized.

ii. Timing Constraints:
   a. Maximum Clock Frequency: This is a key performance indicator, representing the highest achievable clock speed based on the current implementation.
   b. A higher frequency implies that the system can perform faster computations.

iii. Power Consumption:
   a. Estimated Dynamic Power: The amount of power consumed by the design during operation.
   b. Static Power: Power consumed when the FPGA is idle.

These results help to identify potential issues with resource usage or timing and enable optimization

before moving to the next step.

4.1.4 Implementation and Bitstream Generation

After synthesis, the next phase is implementation, which maps the synthesized design onto the actual FPGA hardware resources. This process ensures that the design meets timing constraints and that no resource conflicts arise. Key steps in implementation include:

i. Placement and Routing:

   a. Vivado places the logic elements (LUTs, registers, DSP slices) and routes the signals between them, ensuring that the physical layout is efficient and meets timing requirements.

ii. Timing Analysis:

   a. Vivado checks for any timing violations (e.g., setup and hold time violations) that could cause the design to fail at high clock frequencies.

iii. Bitstream Generation:

   a. Once implementation is successfully completed without violations, Vivado generates a bitstream file. This file contains the configuration data required to program the FPGA with your design.

   b. The bitstream file is then transferred to the FPGA to configure it for execution.

The bitstream file serves as the final step in deploying the design onto the FPGA, where it can be tested with real inputs and outputs.

4.2 SIMULATION AND TESTING

4.2.1 Simulation Setup

Simulations are a critical part of the FPGA design process. They help verify that the design behaves as expected before moving to hardware implementation. This process involves using testbenches to apply inputs and capture outputs, automating the verification of the design's functionality.

i. Functional Simulation:

   a. Functional simulations ensure that the RTL (Register-Transfer Level) design behaves as expected under ideal conditions.

   b. The simulation applies a set of input stimuli to the design (such as operand values for multiplication) and checks the output against the expected results. For this project, functional tests confirm that the Radix-16 Booth Multiplier and the error detection mechanism work correctly.

ii. Post-Synthesis Simulation:

   a. Once the design passes functional simulation, post-synthesis simulation is performed to check whether the design meets timing constraints under real operating conditions.

   b. This simulation uses the synthesized netlist, which includes real timing information after synthesis. It is important for ensuring that the design will work correctly once implemented on the FPGA, accounting for delays, routing, and the actual placement of logic elements.

iii. Testbench Design:

   a. A testbench is created that includes:

      i. Input signal generation (e.g., operand values for A and B).

      ii. Expected output for each test case.

      iii. Simulation of various conditions like error detection and parity mismatch.

      iv. Timing constraints for the clock signals to ensure the system operates within the FPGA’s timing limits.

Simulations help catch functional or timing-related errors early in the design process, significantly

reducing the chances of issues during hardware testing.

4.2.2 Waveform Analysis

The waveform output from simulations is a powerful tool for analyzing the operation of the design. It allows the designer to visually inspect signal transitions over time, ensuring that the system works correctly at every stage.

Key aspects to examine in the waveform analysis include:

i. Booth Encoding:

   a. Ensure that the Booth encoder generates the correct partial products based on the 5-bit window from the multiplier.

   b. Check the encoding transitions in the waveform to ensure that the product terms (M, 2M, 3M, etc.) are generated correctly according to Booth's algorithm.

ii. Partial Product Accumulation:

   a. Verify that the partial products are accumulated correctly, shifted according to the correct bit positions, and added in the correct order.

   b. Examine the signal transitions of the partial product register and the accumulator to ensure there are no errors.

iii. Parity Generation:

   a. Since parity checking is integrated for error detection, it is essential to monitor the parity signals in the waveform to ensure they are being generated and updated correctly throughout the computation process.

   b. Compare the computed parity with the expected parity to confirm that the error detection mechanism is functioning as intended.

iv. Error Detection:

   a. Check that the error detection unit responds correctly to errors, triggering the error flag

when a mismatch between computed and expected parity occurs.

b. If an error is detected, the corresponding error signal should be visible in the waveform.

By closely analyzing the waveforms as shown below in fig.4.2.2, any mismatches or anomalies in the design can be traced back to specific parts of the RTL code, making debugging easier.

Fig.4.2.2. (a)-Radix 4 Simulation

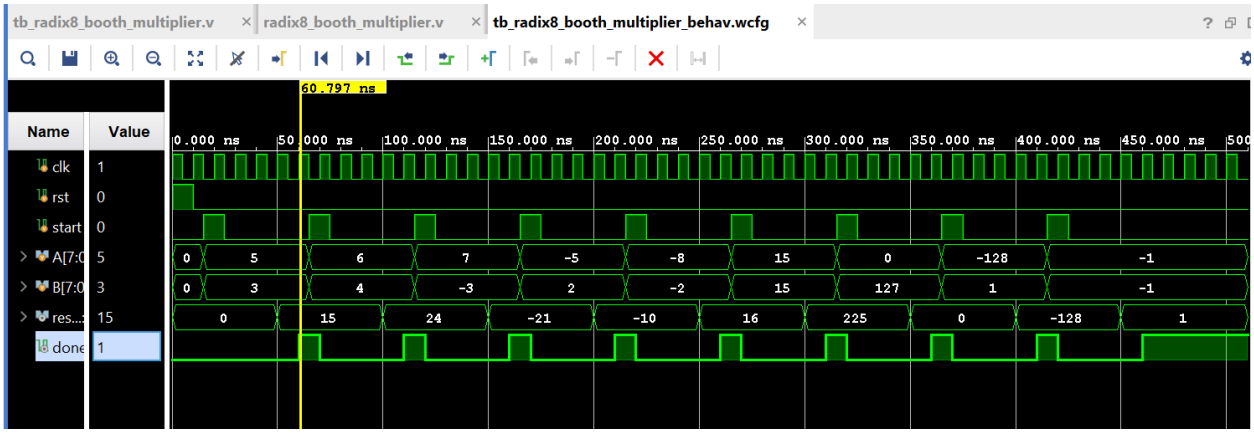

Fig.4.2.2. (b)-Radix 8 Simulation

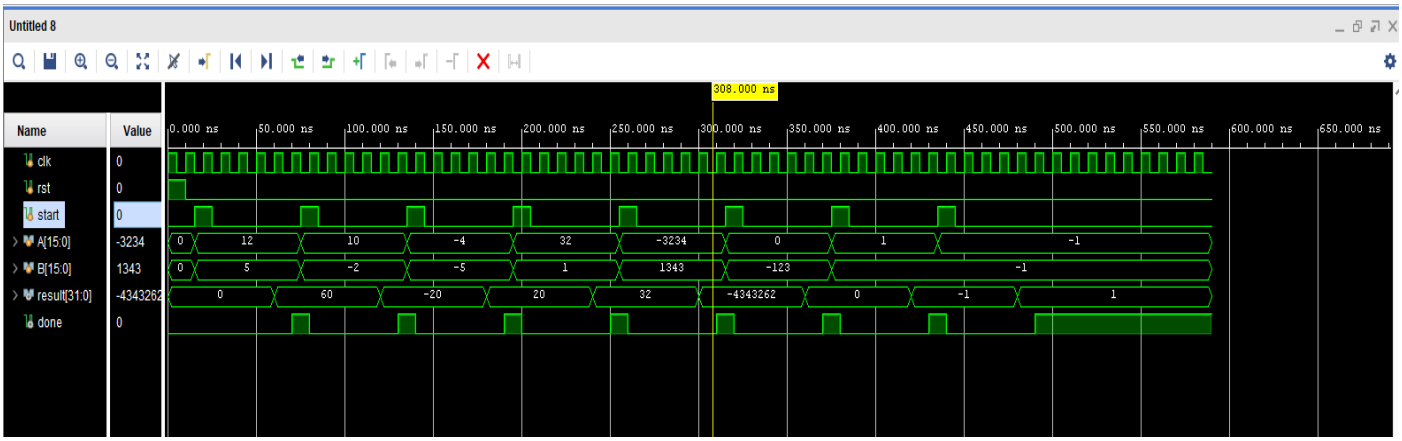

Fig.4.2.2. (c)-Radix 16 Simulation

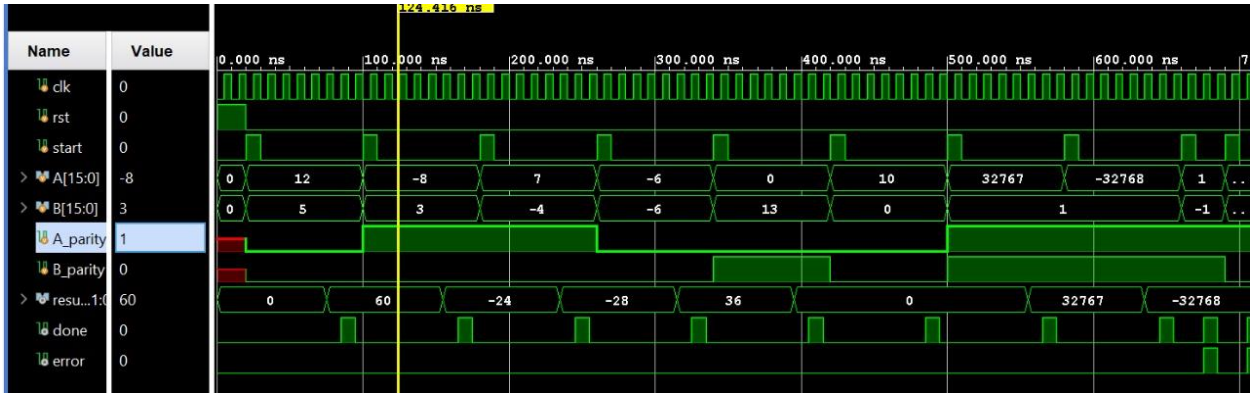

Fig.4.2.2. (d)-Radix 16 with Error Detection and Correction Simulation

4.2.3 Hardware Validation

After the bitstream has been generated and programmed into the FPGA, hardware validation ensures that the design works in a real-world setting.

i. Input Verification:

   a. Manually set operand values (A and B) using the FPGA switches. This mimics real-world input conditions where user inputs are provided via hardware switches.

   b. The design should correctly load 16-bit operands across two cycles (since the switches are 8 bits wide), and once the operands are loaded, the multiplication operation should start automatically.

ii. Output Verification:

   a. Observe the output on the LEDs:

i. Ensure that the 7 least significant bits of the multiplication result are displayed on LEDs LED [7:1].

ii. Check that the error flag (if any errors are detected) is correctly displayed on LED [0].

iii. ILA Probes:

a. Integrated Logic Analyzer (ILA) probes are used to monitor internal signals during hardware testing. This allows for real-time inspection of the internal signals, helping identify issues like improper signal propagation or unexpected delays.

b. ILA probes can be configured to capture critical signals like the partial product accumulator, Booth encoding outputs, error detection signals, and parity generation signals.

c. By observing these internal signals during testing, any discrepancies in the expected behavior can be pinpointed and corrected.

iv. Real-Time Testing:

a. Validate the design under different operating conditions:

i. Test with positive and negative operand values to ensure proper handling of sign extensions and arithmetic operations.

ii. Introduce faults or error conditions (e.g., simulated bit flips) to ensure that the error detection mechanism triggers correctly.

Hardware validation is crucial for ensuring the design works not just in simulations, but also under the real-world conditions and timing constraints that occur on tactual FPGA hardware as shown in fig.4.2.3 (a).

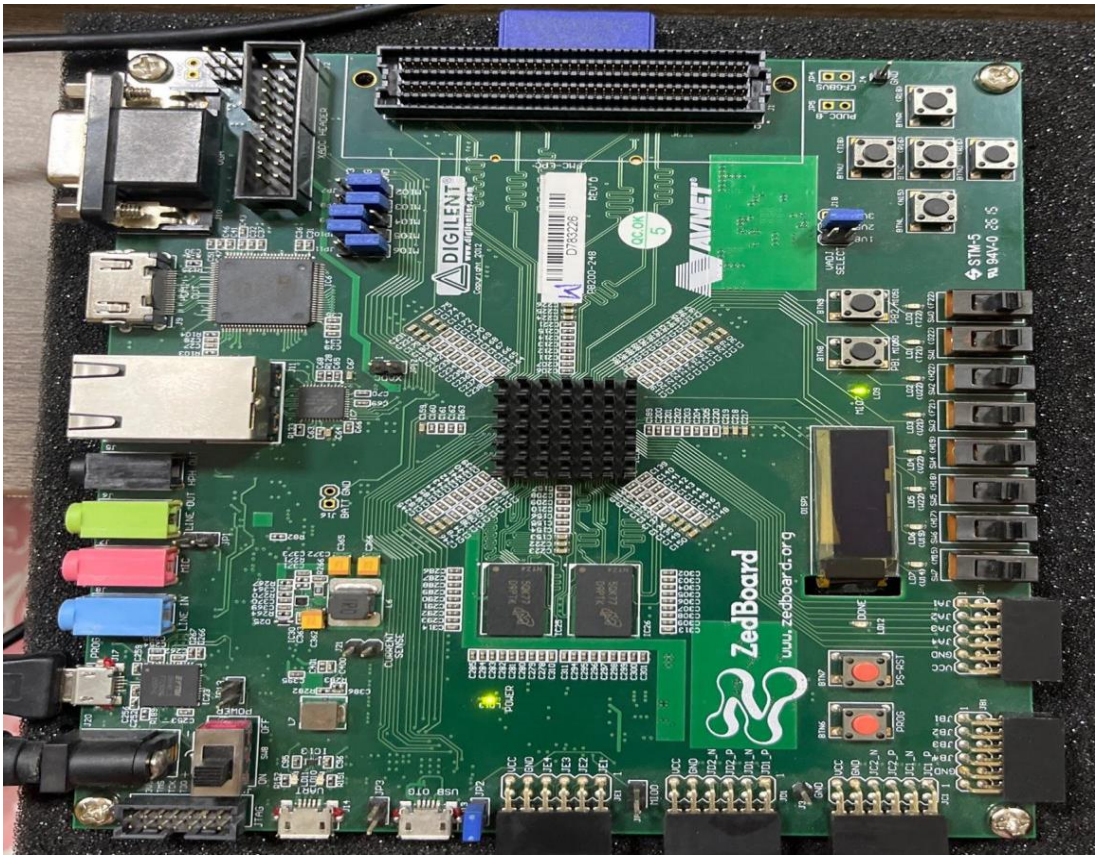

Fig.4.2.3. (a)-Zynq 7020 Zedboard (FPGA Board) Implementation

5. RESULTS AND ANALYSIS

5.1 Performance Analysis

Performance analysis is essential to understanding the efficiency and trade-offs of the Approximate Radix-16 Booth Multiplier compared to traditional exact multipliers. The key performance metrics are:

i. Clock Frequency:

   a. The maximum clock frequency achievable by the design is measured to evaluate the operating speed. A higher clock frequency generally indicates faster processing, though it may impact power consumption and resource utilization.

   b. For this design, the clock frequency is compared with conventional exact multipliers to see if the approximation methods (such as reducing the number of partial products) affect the speed of the system.

ii. Area Usage:

   a. Area utilization is measured in terms of resources like Look-Up Tables (LUTs), flip-flops, and DSP slices.

   b. The design is optimized to use fewer resources than traditional exact multipliers. The approximation techniques and parity-based error detection result in reduced area usage, making the design more efficient for FPGA implementation.

   c. The area savings are quantified and compared with an exact radix-16 multiplier, which would require a larger number of resources to handle full-precision multiplication.

iii. Power Consumption:

   a. Power consumption is another critical metric for evaluating the design's efficiency, especially when deploying on FPGAs with limited power resources.

   b. Power consumption is estimated using tools available in Vivado, and comparisons are made against the power usage of exact multipliers. The results show how approximation techniques, which reduce the number of operations, help lower dynamic power consumption.

iv. Computational Accuracy:

a. Computational accuracy is measured by comparing the results of the approximate multiplier to the exact value of the multiplication. The error margin is calculated, showing the deviation from the correct result.

b. The integration of error detection mechanisms helps in maintaining the accuracy within acceptable limits, allowing for tolerable errors while still achieving significant performance gains in terms of speed and resource usage.

The trade-offs between these metrics as shown in fig.5.1 provide a comprehensive view of how the approximate multiplier operates in terms of speed, resource usage, power efficiency, and accuracy.

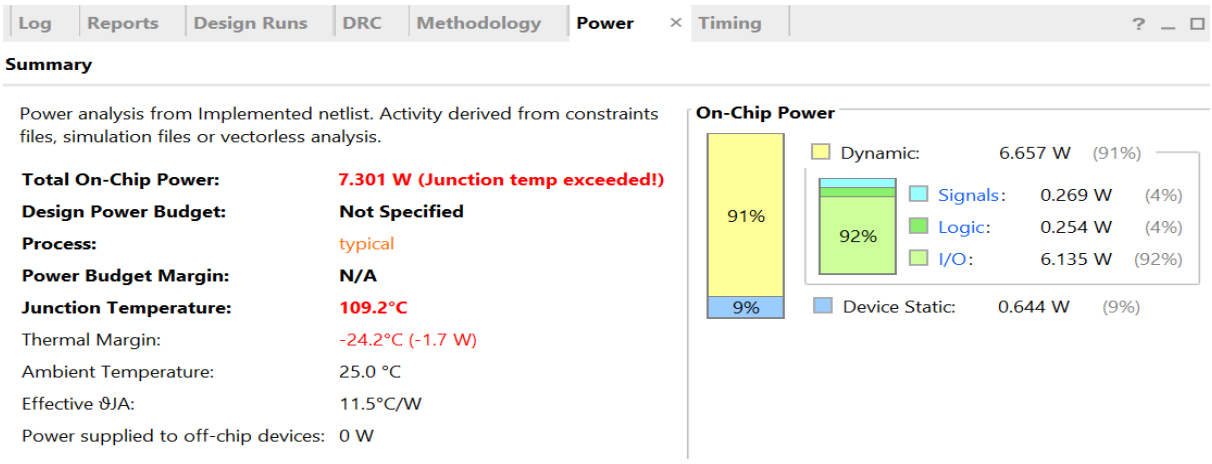

Fig5.1. (a)-Power Analysis of Radix 4

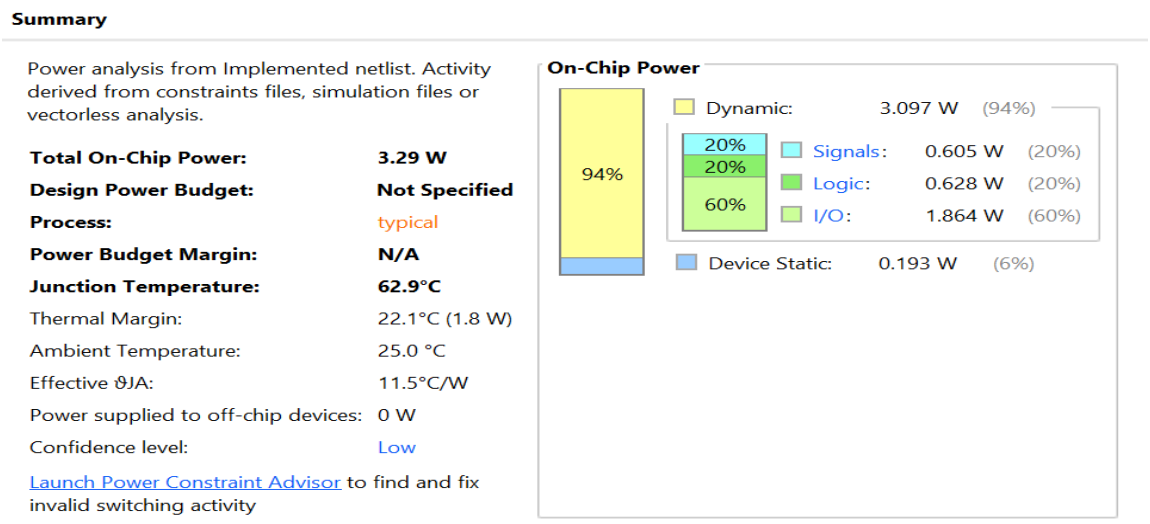

Fig.5.1. (b)-Power Analysis of Radix 8

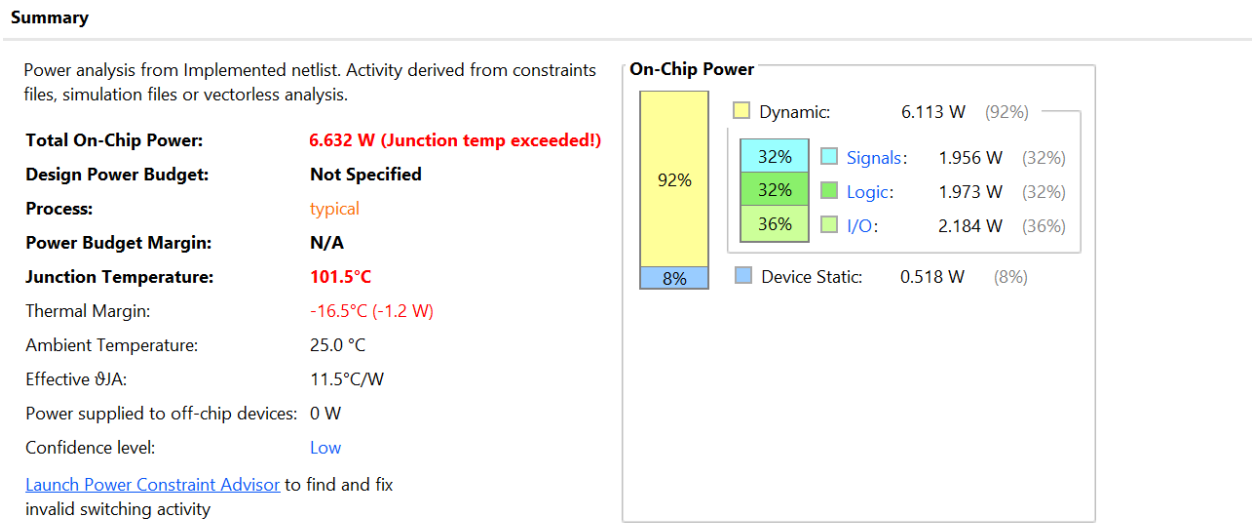

Fig.5.1. (c)-Power Analysis of Radix 16

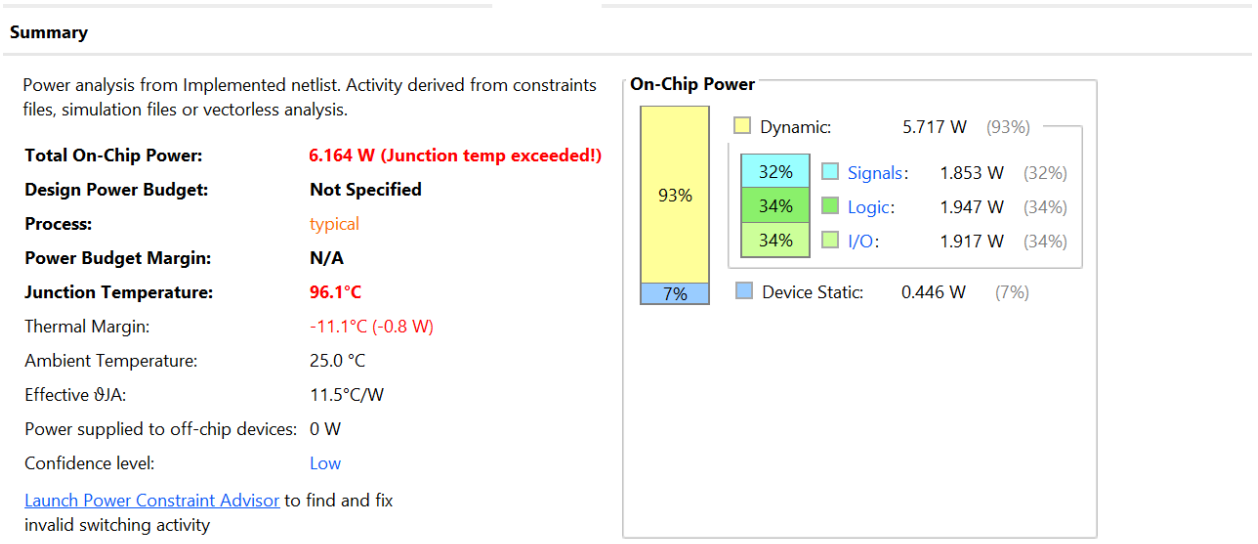

Fig.5.1. (d)-Power Analysis of Radix 16 with Error Detection and Correction

5.2 Error Detection Analysis

The effectiveness of the parity-based error detection (EDC) mechanism is evaluated through a series of fault injection tests, simulating potential bit errors in the computation process. This allows for a thorough assessment of the EDC system's reliability. The following metrics are used to evaluate the EDC approach:

i. Detection Rate:

   a. The detection rate measures how often the error detection mechanism successfully identifies an error when it occurs. A high detection rate indicates that the system is robust and can catch a majority of errors caused by noise or faults in the system.

   b. The detection rate is calculated by comparing the number of faults injected with the number of faults detected by the system during the testing phase.

ii. False Positives:

   a. False positives occur when the error detection mechanism incorrectly flags a normal operation as erroneous. This metric is important for understanding the reliability of the error detection approach.

   b. The rate of false positives is recorded to assess whether the system falsely detects errors that do not actually exist, leading to unnecessary intervention or retransmissions.

iii. Latency to Flag Errors:

   a. Latency is the time delay between when an error occurs and when the error is detected and flagged. This metric is crucial for understanding how quickly the error detection system can react to potential faults.

   b. Lower latency ensures that the system can recover or correct the error faster, minimizing its impact on the overall computation.

The error detection analysis highlights the effectiveness of using parity checking to maintain the integrity of the system while minimizing overhead.

5.3 Area, Speed, Power Trade-Off

A key aspect of this design is the trade-off between area, speed, and power consumption as shown in fig.5.3. The following analyses are performed:

i. Area Savings:

   a. The approximation techniques employed in the Radix-16 Booth Multiplier significantly

reduce the number of operations and partial products, leading to reduced LUT usage and DSP slices.

b. Compared to exact multipliers, this design achieves substantial area savings, which is critical in FPGA-based systems where resources are limited.

ii. Speed vs. Accuracy:

a. Speed is increased by reducing the number of partial products generated during the Booth encoding stage. However, this leads to slight inaccuracies in the final product.

b. The analysis quantifies the trade-off between the speed of multiplication and the accuracy of the result. While the approximation allows for faster multiplication, the error margin remains within acceptable limits due to the error detection mechanism.

iii. Power Efficiency:

a. The power consumption of the approximate multiplier is lower than that of an exact radix-16 multiplier, primarily due to the reduced area and simplified computations.

b. Power consumption is analyzed by considering both dynamic power (from switching activity) and static power (from leakage currents). The results show that the approximate multiplier offers significant power savings, making it more suitable for resource-constrained applications.

iv. Trade-Off Visualization:

a. A comprehensive visualization of the trade-offs is provided, showing the relationship between area, speed, accuracy, and power.

b. The analysis presents how the design choices in approximation affect the overall performance, providing a complete understanding of the design's trade-offs.

By evaluating these trade-offs, the design ensures that the approximate Radix-16 Booth Multiplier offers a well-balanced performance in terms of speed, area, power, and accuracy, making it a suitable choice for applications where performance and resource constraints are critical.

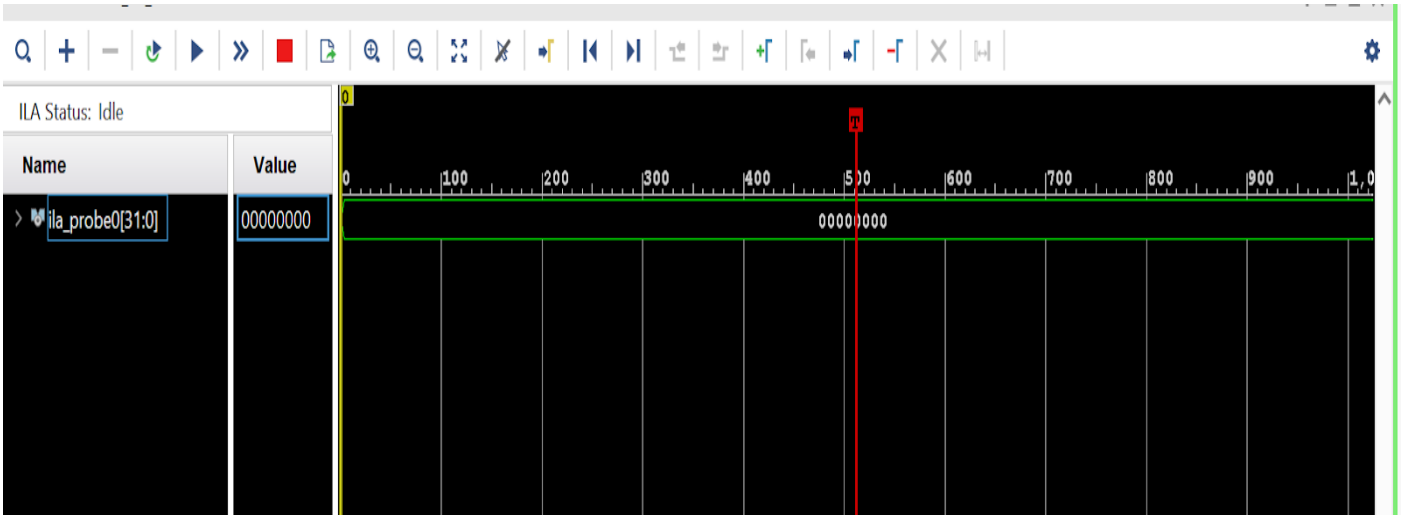

Fig.5.3. (a)-Hardware Implementation Wave Signal Output

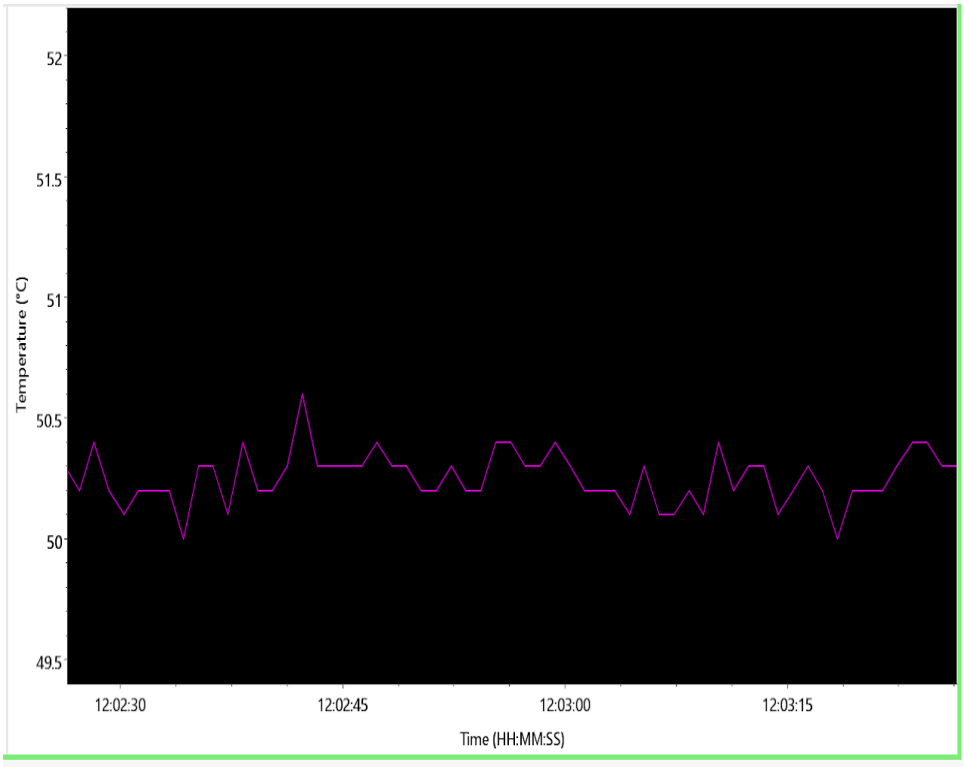

Fig.5.3. (b)-Temperature Analysis of Zynq 7020 board

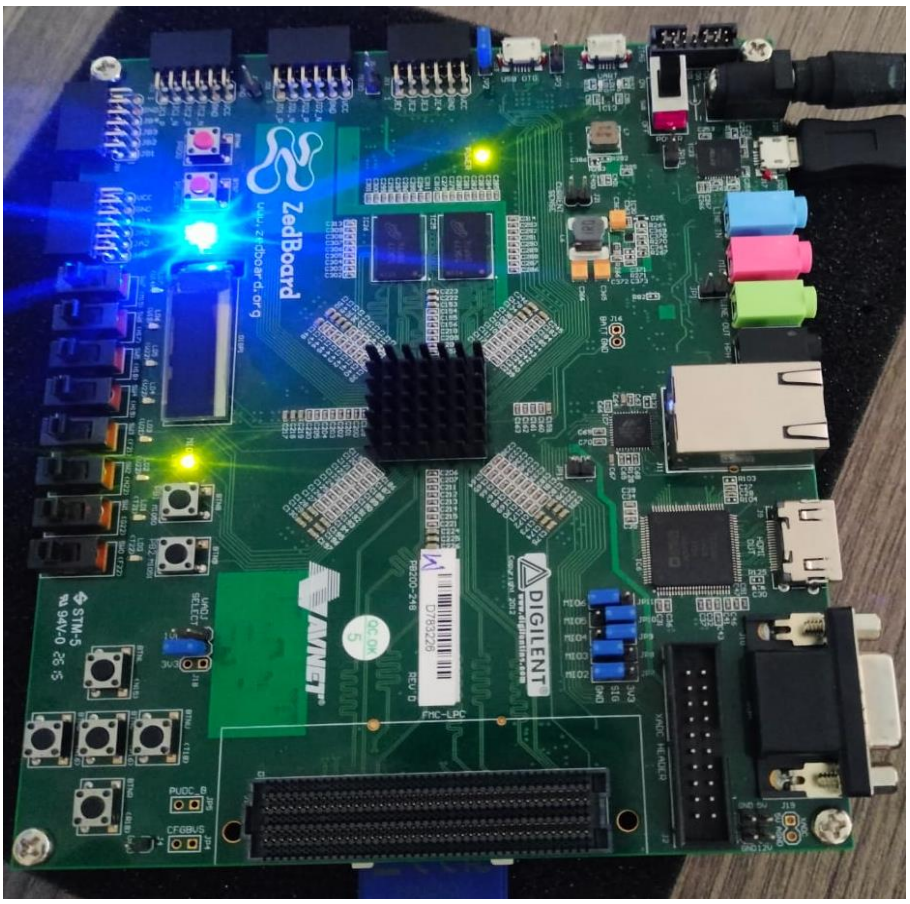

Fig.5.3. (c)-Hardware Implementation Output on Zynq 7020

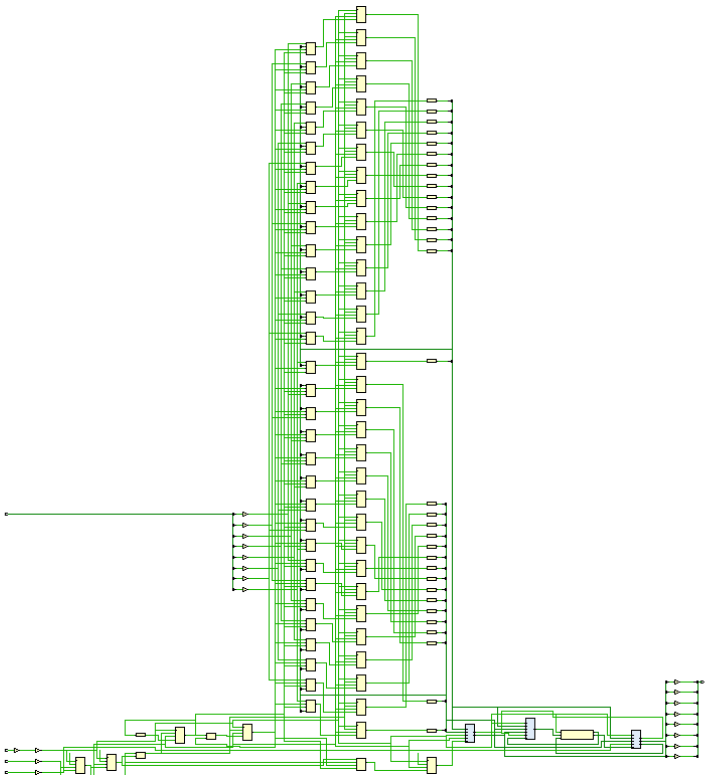

Fig.5.3. (d)-Logic Diagram of FPGA Implementation of Radix 16 with Error Detection and Correction

6.CONCLUSION AND FUTURE WORK

6.1 Conclusion

The project has successfully achieved its goals of designing a high-performance approximate Radix-16 Booth multiplier integrated with a lightweight error detection and correction (EDC) mechanism. The key accomplishments are as follows:

i. Design Success:

   a. The approximate multiplier demonstrates substantial improvements in speed, area, and power efficiency compared to traditional exact multipliers.

   b. By utilizing approximation techniques, the design reduces the number of partial products and enhances computational speed, while still maintaining reasonable accuracy for most practical applications.

ii. Integration of Error Detection:

   a. The lightweight parity-based error detection mechanism ensures that any errors due to faults or bit flips can be detected efficiently, maintaining the system's reliability.

   b. The EDC mechanism adds minimal hardware overhead while providing an effective way to safeguard against catastrophic faults in approximate circuits.

iii. FPGA Implementation:

   a. The FPGA implementation of the design validates the theoretical performance gains in terms of reduced LUT usage, power consumption, and faster computation, confirming that the approximation combined with basic error detection is both viable and practical for real-world applications.

In conclusion, the project demonstrates that approximate computing with integrated error detection is a promising approach for improving the performance of computationally intensive hardware applications while maintaining reliability.

6.2 Future Work

The project lays a solid foundation for future research and design improvements. Some key directions for future work include:

i. Dynamic Approximation Schemes:

   a. Exploring dynamic approximation schemes where the level of approximation can be adjusted based on the error tolerance required by the application, allowing for better flexibility in balancing performance and accuracy.

ii. Advanced Error Correction Codes:

   a. Implementing more advanced error correction codes (ECC) like Hamming or Cyclic Redundancy Check (CRC) can further improve the robustness of the design against errors, enabling the system to recover from errors more effectively and ensuring higher reliability in critical applications.

iii. Support for Wider Operand Sizes:

   a. Extending the design to support wider operand sizes such as 32-bit or 64-bit operands, enabling the multiplier to handle larger numbers and thus making it applicable to a broader range of applications.

iv. ASIC Implementation:

   a. Transitioning the design to an ASIC (Application-Specific Integrated Circuit) implementation for mass production, providing a more efficient, cost-effective solution for high-volume applications.

These advancements could make the design more scalable, versatile, and reliable, broadening its applicability to various industrial, scientific, and consumer applications.

References

[1]R. Orugu, S. Padamata, Y. Kollati, L. Nakka, Y. Nunna, R. S. M. Mamidi, “FPGA Design and Implementation of Approximate Radix-8 Booth Multiplier,” *Proceedings of the 2024 Third International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE)*, 2024.

[2]PG Scholar and Associate Professor, Department of ECE, Vasireddy Venkatadri Institute of Technology, Guntur, AP, India, “Improved 64-Bit Radix-16 Booth Multiplier Based on Partial Product Array Height Reduction,” *Journal Publication*, Jan–Dec 2018.

[3]M. Thomas, “Design and Simulation of Radix-8 Booth Encoder Multiplier for Signed and Unsigned Numbers,” *International Journal for Innovative Research in Science & Technology (IJIRST)*, vol. 1, no. 1, pp. –, Jun. 2014.

[4]Barma Venkata RamaLakshmi et.al “Implementation and Comparison of Radix-8 Booth Multiplier by using 32-bit Parallel prefix adders for High-Speed Arithmetic Applications”. Turkish Journal of Computer and Mathematics Education Vol.12 No.3(2021), 5673-5683.

[5]Y. Guo, H. Sun, and S. Kimura, “Small-area and low-power FPGA based multipliers using approximate elementary modules,” in Proc. 25th Asia South Pac. Design Autom. Conf. (ASP-DAC), Beijing, China, 2020, pp. 599–604.

[6]Muhammad Hamis Haider, Seok-Bum Ko "Booth Encoding-Based Energy Efficient Multipliers for Deep Learning Systems" IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS, VOL. 70, NO. 6, JUNE 2023

[7]Zainab Aizaz "Area and Power Efficient Truncated Booth Multipliers Using Approximate Carry-Based Error Compensation" IEEE transactions on circuits and systems—ii: express briefs, vol. 69, no. 2, February 2022.

[8]Gunho Park, Jaeha Kung, Youngjoo Lee, “Simplified Compressor and Encoder Designs for Low-Cost Approximate Radix-4 Booth Multiplier" IEEE Transactions on circuits and systems—ii: express briefs, vol. 70, no. 3, March 2023.

[9]Tingting Zhang, Weiqiang Liu,Leibo Liu, Jie Han, “Design of Majority Logic-Based

Approximate Booth Multipliers for Error-Tolerant Applications" IEEE Transactions on nanotechnology, vol. 21, 2022.

[10]Basant Kumar Mohanty "Efficient Approximate Multiplier Design Based on Hybrid Higher Radix Booth Encoding" IEEE journal on emerging and selected topics in circuits and systems, vol. 13, no. 1, March 2023

[11]U. Geetalakshmi, M.V. Nageswara Rao “Asic Implementation of 12 Bit Radix-8 Booth Multiplier” International Journal of Recent Technology and Engineering (IJRTE) ISSN: 2277-3878 (Online), Volume-8 Issue-2, July 2019.

[12]Haroon Waris, Weiqiang Liu, Fabrizio Lombardi “AxBMs: Approximate Radix-8 Booth Multipliers for High-Performance FPGA Based Accelerators” IEEE transactions on circuits and systems—II: express briefs, vol. 68, no. 5, May 2021.

[13]Bipul Boro, K. Manikantta Reddy, Y.B. Nithin Kumar, M.H. Vasantha “Approximate Radix-8 booth multiplier for low power and high-speed applications” Microelectron.

[14]Yong Hyeon Kwon, Jae Wook Jeon “Comparison of FPGA Implemented Sobel Edge Detector and Canny Edge Detector” 2020 IEEE International Conference on Consumer Electronics - Asia (ICCE Asia).

[15]Shuchi Nagaria, Anushka Singh, Vandana Niranjan “Efficient FIR filter design using Booth Multiplier for VLSI applications”. 2018 International Conference on Computing, Power and Communication Technologies (GUCON) Galgotias University, Greater Noida, UP, India. Sep 28-29, 2018.

[16] Farzad Farshchi, Muhammad Saeed Abrishami, and Sied Mehdi Fakhraie "New Approximate Multiplier for Low Power Digital Signal Processing" The 17th CSI International Symposium on Computer Architecture & Digital Systems (CADS 2013)

[17]T. Satish Kumar, Dr. G. ManmadhaRao “An approach of Modified Radix-8 Booth Multiplier using Verilog”. International Journal of Advance Research in Science and Engineering.

[18]S. Venkatachalam, E. Adams, H. J. Lee, and S.-B. Ko, “Design and analysis of area and power efficient approximate booth multipliers,” IEEE Trans. Comput., vol. 68, no. 11, pp.